Hallucinating our way to AI-backed documentation

PART I - THEORY

Who am I

- AI Solutions Architect at Cennso
- Independent open-source maintainer - brainfart.dev
- Maintainer of AsyncAPI spec and generator
- Community Servant at Open Source Europe

Lukasz Gornicki
Pushing the boundaries in AsyncAPI
Initiative and Open Source

The big words

🤖 **Agentic**

🧠 **LLM**

🔍 **RAG**

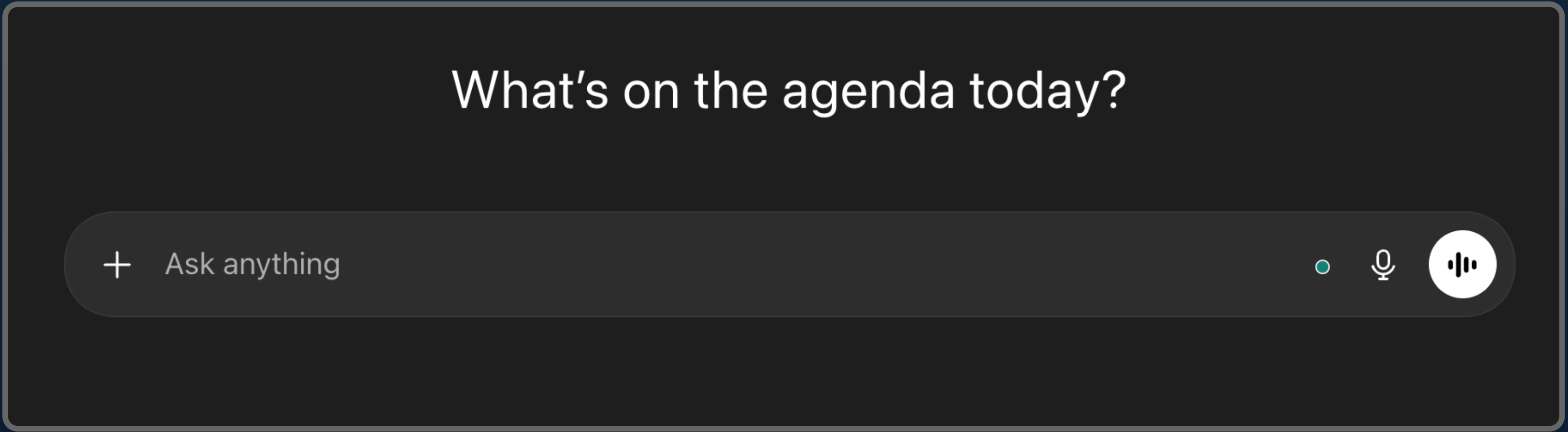
🔌 **MCP**

🦮 **Skills**

What is Agentic AI

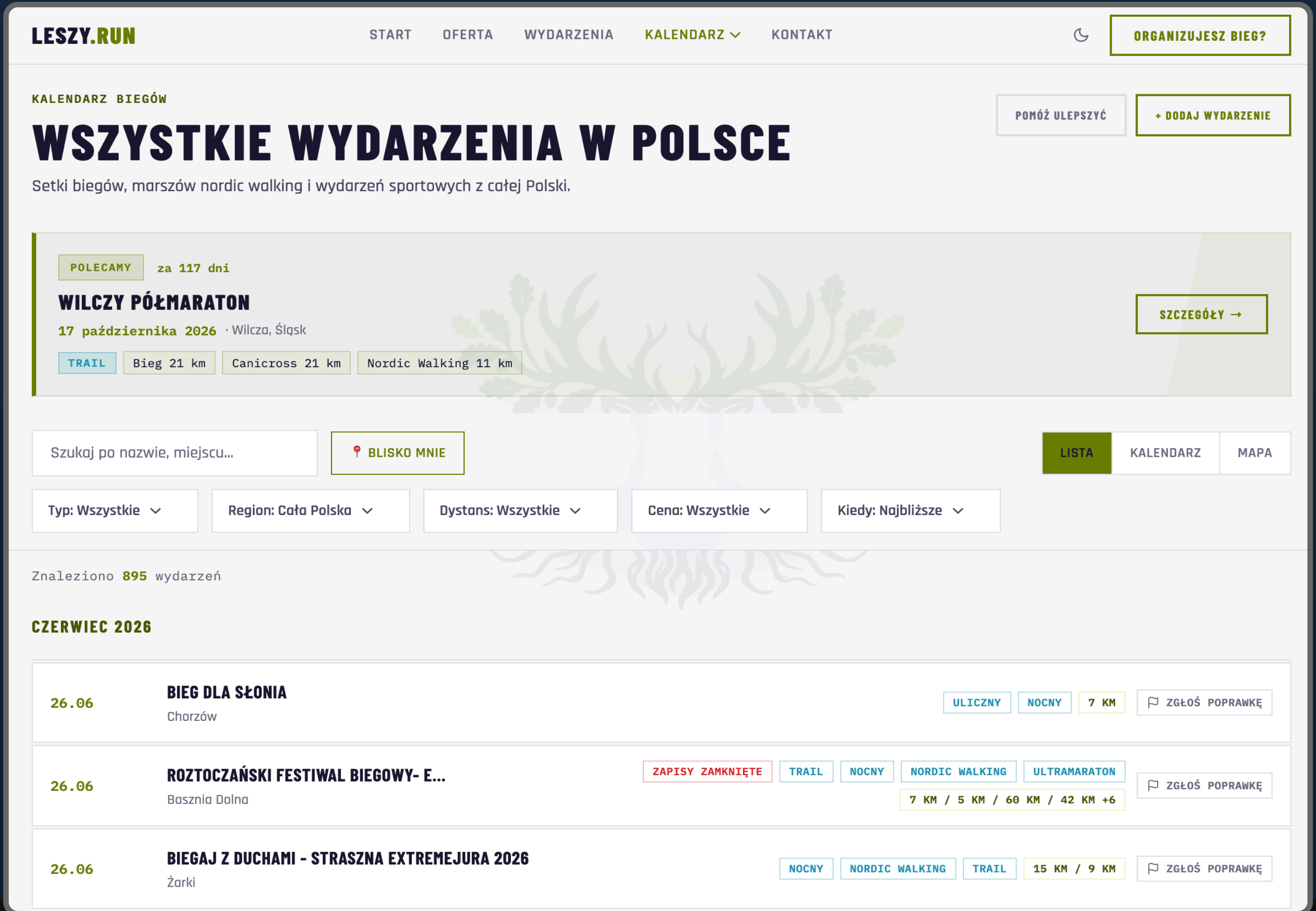
AI without copy/paste

Chat AI:



Agent mode

AI without copy/paste



The big words

🤖 ~~Agentic~~

🧠 **LLM**

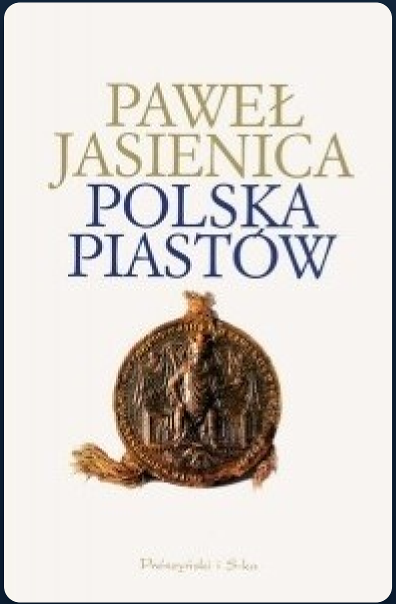
🔍 **RAG**

🔌 **MCP**

🦮 **Skills**

What is LLM

You don’t know what the LLM knows about the Piast dynasty



Polska Piastów – Paweł Jasienica

Drapieżny ród Piastów – Sławomir Leśniewski

You don’t know where the LLM gets its knowledge about AsyncAPI

can I use protobuf with AsyncAPI

Yes. **Protocol Buffers (Protobuf)** can absolutely be used with **AsyncAPI**, though they serve different purposes:

- **AsyncAPI** describes your event-driven API (channels, messages, operations, servers, etc.).
- **Protobuf** defines the binary serialization format and message schema.

They complement each other well.

Option 1: Reference `.proto` schemas (recommended)

You can define your message in a `.proto` file:

proto
```
syntax = "proto3";

package user;

message UserCreated {
  string id = 1;
  string email = 2;
  string name = 3;
}
```

Then reference it from your AsyncAPI document:

YAML
```yaml
asyncapi: 3.0.0

channels:
  user.created:
    address: user.created
    messages:
      UserCreatedMessage:
        payload:
          schemaFormat: application/vnd.google.protobuf;version=3
          schema:
            $ref: "./schemas/user.proto#/UserCreated"
```

The important field is:

YAML
```yaml
schemaFormat: application/vnd.google.protobuf;version=3
```

AsyncAPI

Local-First

Ask AsyncAPI

Answers about the AsyncAPI spec, concepts, tooling, and community — grounded in the official docs, with sources you can verify.

💜 **AsyncAPI** is built by people with passion, not large corporate funding. **Support it**

YOU

can I use protobuf with AsyncAPI

ASYNCAPI KNOWLEDGE

Yes. AsyncAPI can describe Protobuf payloads. Do this by using Protobuf as the payload schema format and embedding the Protobuf schema itself as a string.

- How it works:
  - In the payload, set payload.schemaFormat to a Protobuf media type (e.g., application/vnd.google.protobuf;version=3).
  - Put the actual Protobuf definition in payload.schema as a string.
  - Do not use $ref to reference parts of the Protobuf schema; the Protobuf schema must be included directly as text.
- Example (YAML):

```yaml
payload:
  schemaFormat: application/vnd.google.protobuf;version=3
  schema: "syntax = \"proto3\"; message User { string id = 1; string name = 2; }"
```

Sources:

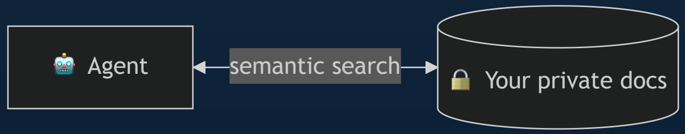
- AsyncAPI define-payload — Schema formats: https://www.asyncapi.com/docs/concepts/asyncapi-document/define-payload

Blind spot

How can the LLM know about your closed-source product with docs behind a login page?

It can't – just a void

RAG



The big words

~~🤖 Agentic~~

~~🧠 LLM~~

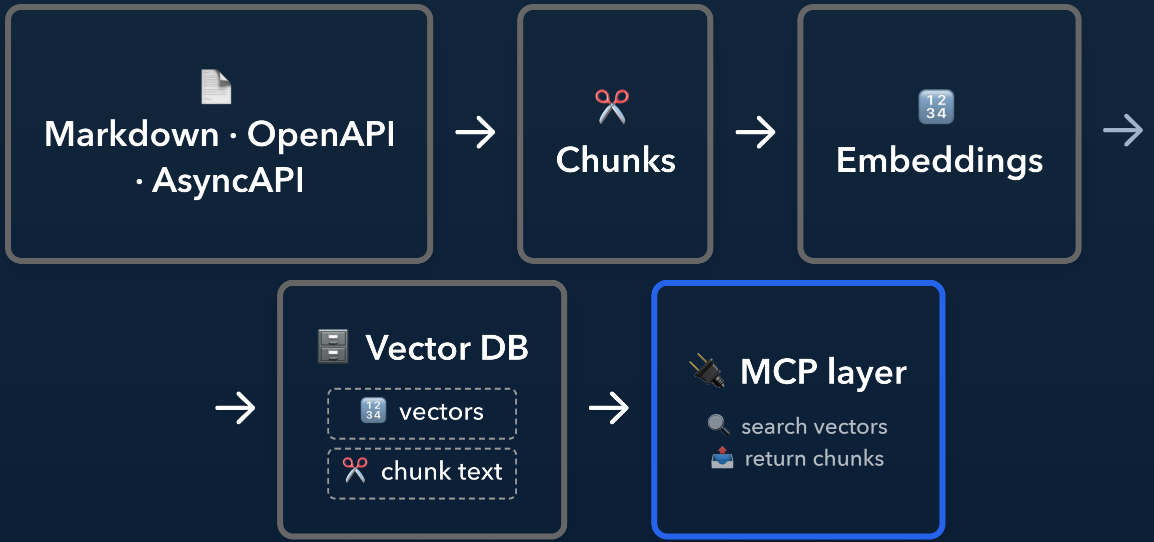
🔍 RAG

🔌 MCP

🦮 Skills

What is RAG

Semantic search over your own content



Chunks

One doc → many structure-aware chunks

📝 Prose


```json
{
  "chunk_id": "c4ce5efe…382289e",
  "chunk_type": "prose",
  "content": "## Multiple channels with single message when reply address is known\n\nThe request/reply pattern can also be implemented over multiple channels with a single message…",
  "line_start": null,
  "metadata": {
    "source_url": "https://www.asyncapi.com/docs/concepts/asyncapi-document/reply-info"
  },
  "source_file": ".opencrane/llmstxt/llms-full.txt",
  "source_name": "asyncapi-website",
  "token_count": 74
}
```

Chunks

📋 List item


```json
{
  "chunk_id": "43ca503b…92cd7ef",
  "chunk_type": "list_item",
  "content": "# … AsyncAPI document generation process\n1. The **Generator** receives the **AsyncAPI Document** as input.",
  "line_start": null,
  "metadata": {
    "breadcrumb_path": "…/asyncapi-document AsyncAPI document generation process",
    "depth": 0,
    "list_id": "7222577c705f7892",
    "list_style": "ordered",
    "parent_item_id": null,
    "position": 1,
    "sibling_ids": ["9714fd50…d554292", "b9728c45…fe3aca0", "0616eda6…59fbd81", "bc681ae6…eeea7a4", "537dc8cb…f176ba6"],
    "sibling_previews": ["2. The **Generator** sends to…", "3. The **Parser** validates t…", "4. If the **Parser** determin…", "5. At this point, the **Gener…", "6. The **Template Context** i…"],
    "source_url": "https://www.asyncapi.com/docs/tools/generator/asyncapi-document",
    "total_siblings": 6
  },
  "source_file": ".opencrane/llmstxt/llms-full.txt",
  "source_name": "asyncapi-website",
  "token_count": 37
}
```

Chunks

🗂️ JSON Schema


```json
{
  "chunk_id": "1fcc25de…395bb4",
  "chunk_type": "json_schema",
  "content": {
    "description": "A unique id representing the application.",
    "format": "uri",
    "type": "string"
  },
  "line_start": null,
  "metadata": {
    "breadcrumb_path": "properties.id",
    "logical_parent": "root",
    "neighbor_chunks": ["bdf39a6c…294dec3", "6dad8d11…1c2c319", …],
    "original_format": "yaml",
    "property_name": "id",
    "property_path": "id",
    "schema_element": "properties",
    "schema_title": "AsyncAPI 3.1.0 schema.",
    "schema_type": "json_schema",
    "schema_version": "http://json-schema.org/draft-07/schema",
    "source_url": "https://github.com/asyncapi/website/blob/master/config/3.1.0.json"
  },
  "source_file": ".opencrane/llmstxt/llms-full.txt",
  "source_name": "asyncapi-json-schema",
```

The big words

🤖 ~~Agentic~~

🧠 ~~LLM~~

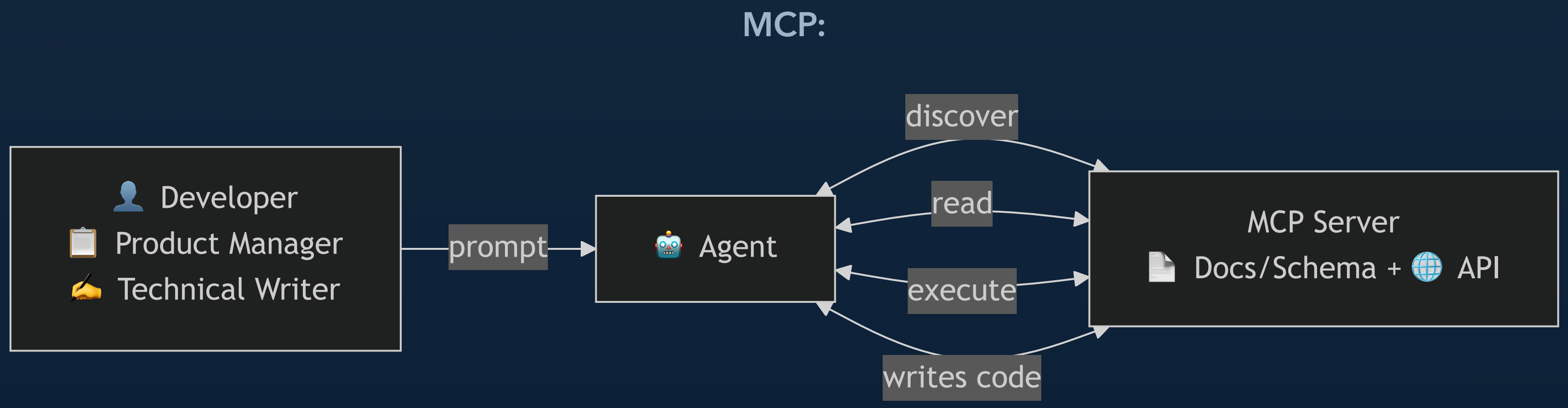
🔍 ~~RAG~~

🔌 **MCP**

🦮 **Skills**

What is MCP

API for AI agents



Behind MCP

What is MCP exposing to the agent

RAG:

Local (uvx):

```json
{
  "mcpServers": {
    "asyncapi-knowledge": {
      "command": "uvx",
      "args": ["asyncapi-knowledge-mcp==0.0.4"]
    }
  }
}
```

Hosted (HTTP):

```json
{
  "mcpServers": {
    "asyncapi-knowledge": {
      "url": "https://derberg-asyncapi-knowledge-mcp.hf.space/http"
    }
  }
}
```

The big words

🤖 ~~Agentic~~

🧠 ~~LLM~~

🔍 ~~RAG~~

🔌 ~~MCP~~

🦮 **Skills**

Skills

Knowledge & tools aren't enough – the agent needs your playbook

🦮 **off the leash** → capable, but does whatever it wants

🦮 **on the leash** → the same trick, your way, every time

Anatomy of a Skill

A `SKILL.md` the agent loads on demand

```
---
name: write-asyncapi-contract
description: Use when drafting, reviewing, or writing an AsyncAPI contract
---

1. Look up the current AsyncAPI v3 spec via the asyncapi-knowledge MCP — never guess from memory
2. Define servers with their protocol and bindings (Kafka, MQTT, AMQP, …)
3. Model channels and their parameters — where messages flow
4. Add operations (send / receive) that reference those channels
5. Define message payloads as reusable JSON Schemas under components
6. Check syntax against the spec via the MCP, then validate (asyncapi validate)
```

Hardening

A skill is never finished – you train it every day

Putting it together

🤖 **Agentic** = does the work for you

🧠 **LLM** = how it thinks

🔍 **RAG** = what it knows

🔌 **MCP** = what it can use

🦮 **Skills** = how repeatable it is

HARNESS

Links – End of Part I

- 🏗️ https://github.com/derberg/OpenCrane/
- 🧩 https://github.com/derberg/asyncapi-knowledge-mcp
- 📊 https://github.com/derberg/eval-bench
- 🖥️ https://github.com/derberg/ai-rag-presentation
- 📄 https://github.com/derberg/ai-rag-presentation/blob/main/slides-export.pdf